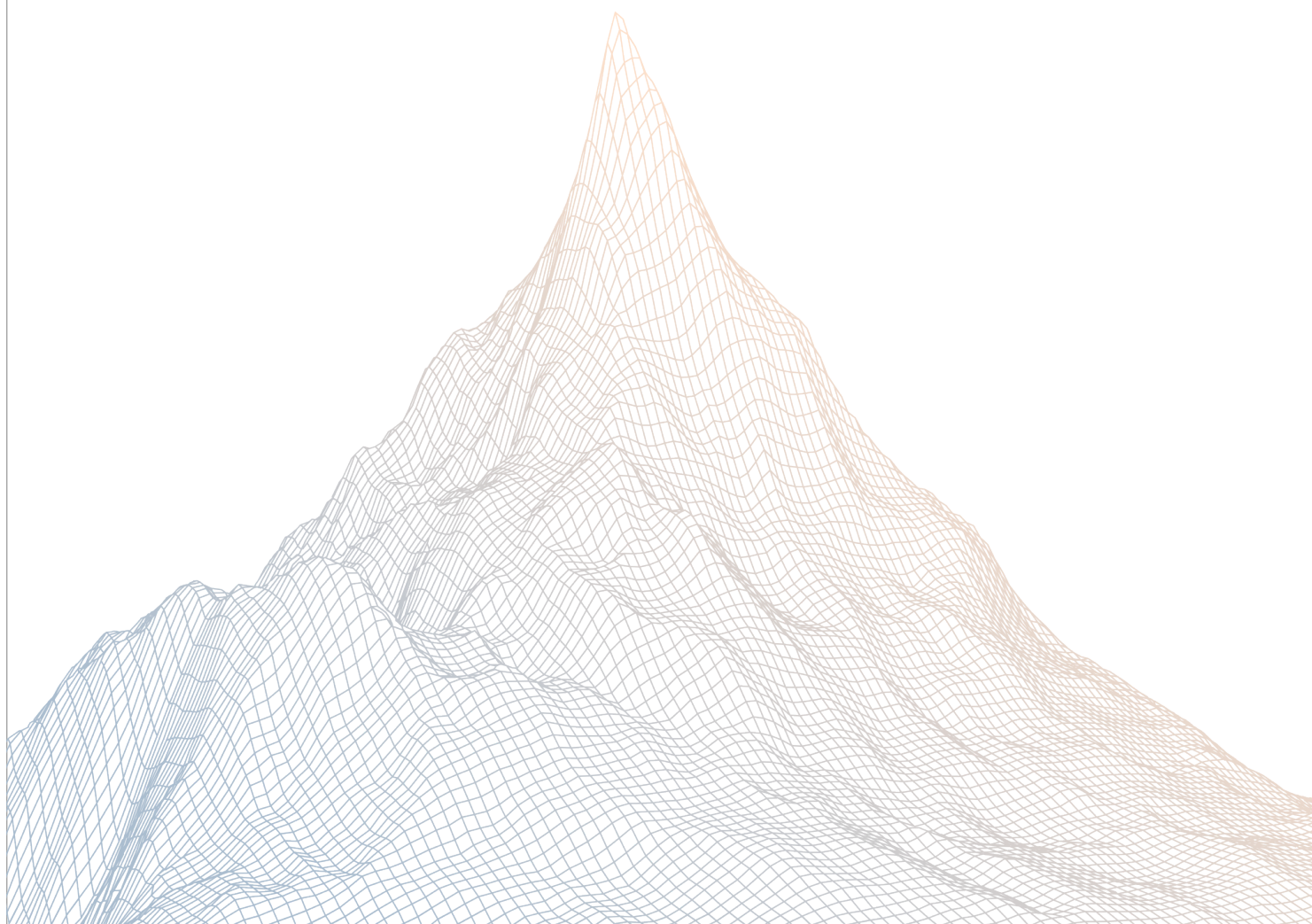


Hyperlink

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

May 29th to June 1st, 2026

AUDITED BY:

adriro
ether_sky

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About HyperLink	4
2.2	Scope	4
2.3	Audit Timeline	8
2.4	Issues Found	8
3	Findings Summary	8
4	Findings	9
4.1	Medium Risk	10
4.2	Low Risk	11
4.3	Informational	18

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About HyperLink

Privacy-preserving prime broker protocol for Hyperliquid using AWS Nitro Enclaves with HyperEVM smart contracts managing deposits, withdrawals, and state verification. Also includes hIHYPE, a liquid staking token for HYPE.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts
---------------	-----------

Repository	https://github.com/hyperlink-xyz/contracts.git
-------------------	---

Commit Hash	71d80a5a99fc2ec3de5f11af8bbcf48a70992a3c
--------------------	--

Files	Diff from 97a700476392aa6ab344dd68adb678067514a73a
--------------	--

Target	contracts
---------------	-----------

Repository	https://github.com/hyperlink-xyz/contracts/pull/46
-------------------	---

Commit Hash	ec679df522902ec91cf31451402219c118729dac
--------------------	--

Files	Diff from 9285a33024c96853a7e761cef66a6a9bd689464f
--------------	--

Target	contracts
---------------	-----------

Repository	https://github.com/hyperlink-xyz/contracts/pull/87
-------------------	---

Commit Hash	71d80a5a99fc2ec3de5f11af8bbcf48a70992a3c
--------------------	--

Files	Diff from bb264ce215789ade6a5502f8502302d812891fb5
--------------	--

Target	contracts
---------------	-----------

Repository	https://github.com/hyperlink-xyz/contracts/pull/89
-------------------	---

Commit Hash	71d80a5a99fc2ec3de5f11af8bbcf48a70992a3c
--------------------	--

Files	Diff from 0114693a22299416629201dd62d3d6362c7fd5fa
--------------	--

Target	contracts
---------------	-----------

Repository	https://github.com/hyperlink-xyz/contracts/pull/90
-------------------	---

Commit Hash	8a2ec652eee62c3a2b0ebc79d4d486e409381807
--------------------	--

Files	Diff from 0c25565f6bebdb9059f16a4add3641e1d49314e6
--------------	--

Target	contracts
---------------	-----------

Repository	https://github.com/hyperlink-xyz/contracts/pull/91
-------------------	---

Commit Hash	6eb40b5c675f94b4957bf2ea481e1d6664ffc845
--------------------	--

Files	Diff from be41a3b04d77e3fc1476bf9a71e404e313fbcc36
--------------	--

Target	contracts
---------------	-----------

Repository	https://github.com/hyperlink-xyz/contracts/pull/92
-------------------	---

Commit Hash	f8216c7921c1e7d8f0cc561a2f16bd75c2bd9403
--------------------	--

Files	Diff from 14393cbbf484a097c0718620fd5273ca901cbef4
--------------	--

Target	contracts
---------------	-----------

Repository	https://github.com/hyperlink-xyz/contracts/pull/93
-------------------	---

Commit Hash	71d80a5a99fc2ec3de5f11af8bbcf48a70992a3c
--------------------	--

Files	Diff from 78a463bf6d856b9e3b5a4679db9ba3952bb54e4e
--------------	--

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/hyperlink-xyz/contracts.git
-------------------	---

Commit Hash	e110574ad37f006ae83a16c06e7b04201c458ed4
--------------------	--

Files	Diff from 97a700476392aa6ab344dd68adb678067514a73a
--------------	--

Target	PR-87 Mitigation Review
---------------	-------------------------

Repository	https://github.com/hyperlink-xyz/contracts/pull/87
-------------------	---

Commit Hash	74d4fe9356fd3b34e890debc8fd4fd00274ab20
--------------------	---

Files	Diff from bb264ce215789ade6a5502f8502302d812891fb5
--------------	--

Target PR-89 Mitigation Review

Repository <https://github.com/hyperlink-xyz/contracts/pull/89>

Commit Hash 458d74cc5f5cf9b354ef13870f20a0bbc7aa5041

Files Diff from 0114693a22299416629201dd62d3d6362c7fd5fa

Target Mitigation Review: PR-91

Repository <https://github.com/hyperlink-xyz/contracts/pull/91>

Commit Hash ff257d6a826cc65a21040bd314bd1b2538fd0672

Files Diff from be41a3b04d77e3fc1476bf9a71e404e313fbcc36

Target PR-93 Mitigation Review

Repository <https://github.com/hyperlink-xyz/contracts/pull/93>

Commit Hash 11f9182ff7d64434089f4b2ffeac0d4b28027f06

Files Diff from 78a463bf6d856b9e3b5a4679db9ba3952bb54e4e

2.3 Audit Timeline

May 29, 2026	Audit start
June 1, 2026	Audit end
June 2, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	3
Informational	9
Total Issues	13

3

Findings Summary

ID	Description	Status
M-1	Reappearance of the shadow staking floor from issue #29	Acknowledged
L-1	Unnecessary check in delegateUndelegated function	Resolved
L-2	The spot balance change should be recorded in the decreaseReservedLstWithdrawal function	Resolved
L-3	Releasing the slashed reserve is possible before slashing is completed	Resolved
I-1	Update PR 90	Acknowledged
I-2	PR 87 and PR 91 should be considered in PR 46	Acknowledged
I-3	1e18 can be returned when the total supply is 0	Acknowledged
I-4	Unnecessary check in the _batchClaimableHypeAmount function	Acknowledged
I-5	Unnecessary check in the decreaseReservedLstWithdrawal function	Resolved
I-6	There is an unnecessary state fetch in the _releaseSlashedReserve function	Resolved
I-7	Pending-inclusive NAV is redundant under normal accounting	Acknowledged
I-8	Priority-fee reserve attribution can mask concurrent undelegated drift	Acknowledged
I-9	Unbounded backward scan in batch refresh enables gas DoS	Acknowledged

4

Findings

4.1 Medium Risk

A total of 1 medium risk findings were identified.

[M-1] Reappearance of the shadow staking floor from issue #29

SEVERITY: Medium

IMPACT: High

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

`stakingAccountBalance()` now returns `Math.max(backing, totalStaked)`, where `backing` is derived from the live `DelegatorSummary` precompile state and `totalStaked` comes from the relay-maintained `totalStakedHype` shadow counter.

This reintroduces the exchange-rate floor discussed in issue #29. If validator-side backing decreases because of slashing, failed Core staking actions, or accounting drift, the live balance can fall while the EVM-side shadow counter remains stale at a higher value. Using `Math.max()` then keeps `totalBalance()` and `exchangeRate()` priced from the stale shadow value instead of the authoritative backing.

As previously described, withdrawal batches processed while the exchange rate is overstated can redeem more than their pro-rata share of real backing, shifting losses to remaining `hlHYPE` holders. Depositors during the same window can also be under-minted.

Recommendations:

Confirm whether reintroducing `Math.max(backing, totalStaked)` was accidental or intentional. If accidental, remove the shadow-counter floor and price shares from live backing. If intentional, document the reason for accepting the issue #29 tradeoff and the operational conditions that make the shadow floor safe.

HyperLink Acknowledged. We are closing the pending-inclusive NAV / shadow-floor branch for launch. The launch path keeps reserved-only live backing and avoids reintroducing the shadow staking floor.

4.2 Low Risk

A total of 3 low risk findings were identified.

[L-1] Unnecessary check in delegateUndelegated function

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVault.sol](#)

Description:

The totalPendingWithdrawal has already been excluded from undelegated. For example, assume there are no pending withdrawals, so totalPendingWithdrawal = 0, and undelegated = 1000. If totalReservedLstWithdrawalHype = 800, then only 200 can be delegated. Shouldn't it be possible to delegate the full 1000? Therefore below checks is not needed.

```
function delegateUndelegated(address validator, uint64 weiAmount)
    external onlyOperator whenNotPaused {
        require(weiAmount > 0, ZeroAmount());
        require(block.number > lastDelegationChangeBlockNumber,
            CannotChangeDelegationUntilNextBlock());
        require(whitelistedValidators[validator],
            ValidatorNotWhitelisted(validator));
        require(tokenRelay != address(0), TokenRelayNotSet());

        ▷ PrecompileLib.DelegatorSummary memory summary
          = PrecompileLib.delegatorSummary(tokenRelay);
        ▷ (, uint256 reservedEvm)
          = IHyperlinkTokenRelay(tokenRelay).getLstAccountingState();
        ▷ uint256 pendingWithdrawalEvm
          = summary.totalPendingWithdrawal.to18Decimals();
        ▷ uint256 undelegatedEvm = summary.undelegated.to18Decimals();
        ▷ uint256 reservedUndelegatedEvm = reservedEvm > pendingWithdrawalEvm ?
          reservedEvm - pendingWithdrawalEvm : 0;
```

```

    > uint256 unreservedEvm = undelegatedEvm > reservedUndelegatedEvm ?
      undelegatedEvm - reservedUndelegatedEvm : 0;
    > if (weiAmount.to18Decimals() > unreservedEvm)
      revert Errors.InsufficientBalance();

    IHyperlinkTokenRelay(tokenRelay).delegateUndelegated(validator,
      weiAmount);

    lastDelegationChangeBlockNumber = block.number;
  }

```

Recommendations:

```

function delegateUndelegated(address validator, uint64 weiAmount)
  external onlyOperator whenNotPaused {
    require(weiAmount > 0, ZeroAmount());
    require(block.number > lastDelegationChangeBlockNumber,
      CannotChangeDelegationUntilNextBlock());
    require(whitelistedValidators[validator],
      ValidatorNotWhitelisted(validator));
    require(tokenRelay != address(0), TokenRelayNotSet());
    address manager = stakingVaultManager;
    if (manager == address(0)) revert Errors.InvalidAddress();
    if (IStakingVaultManager(manager).validator() != validator)
      revert Errors.InvalidAddress();

```

```

    PrecompileLib.DelegatorSummary memory summary
    = PrecompileLib.delegatorSummary(tokenRelay);
    (, uint256 reservedHype, uint256 priorityFeeReserveHype) =
      IHyperlinkTokenRelay(tokenRelay).getLstAccountingState();

```

```

    uint256 pendingWithdrawalHype =
      summary.totalPendingWithdrawal.to18Decimals();

```

```

    uint256 reservedUndelegatedHype =

```

```

      reservedHype > pendingWithdrawalHype ? reservedHype - pendingWithdrawa
        lHype : 0;

```

```

    uint256 undelegatedHype = summary.undelegated.to18Decimals();

```

```

    uint256 unreservedUndelegatedHype =

```

```

      undelegatedHype > reservedUndelegatedHype ? undelegatedHype -
        reservedUndelegatedHype : 0;

```

```
uint256 priorityReserveHype =  
  
    unreservedUndelegatedHype > priorityFeeReserveHype ? priorityFeeReserv  
        eHype : unreservedUndelegatedHype;  
  
    undelegatedHype > priorityFeeReserveHype ? priorityFeeReserveHype :  
        undelegatedHype;  
  
    unreservedUndelegatedHype -= priorityReserveHype;  
    undelegatedHype -= priorityReserveHype;  
  
    if (weiAmount.to18Decimals() > unreservedUndelegatedHype) revert Errors.  
        InsufficientBalance();  
    if (weiAmount.to18Decimals() > undelegatedHype) revert  
        Errors.InsufficientBalance();  
  
    IHyperlinkTokenRelay(tokenRelay).delegateUndelegated(validator,  
        weiAmount);  
  
    lastDelegationChangeBlockNumber = block.number;  
}
```

HyperLink: Resolved with [@74d4fe9356 ...](#)

Zenith: Verified.

[L-2] The spot balance change should be recorded in the decreaseReservedLstWithdrawal function

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVault.sol](#)

Description:

When the decreaseReservedLstWithdrawal function is called, the HYPE is moved from the token relay to the Vault's spot balance.

```
function claimUnstakedToVault(uint64 weiAmount)
    external onlyStakingVault whenNotPaused {
    _requireMinGas(1);
    CoreWriterLib.sendAsset(
        $.stakingVault,
        address(0),
        HLCConstants.SPOT_DEX,
        HLCConstants.SPOT_DEX,
        HLCConstants.hypeTokenIndex(),
        weiAmount
    );
}
```

However, this balance change is not recorded

Recommendations:

```
function decreaseReservedLstWithdrawal(uint256 amountWei) external {
    if (msg.sender != stakingVaultManager) revert OnlyStakingVaultManager();
    require(tokenRelay != address(0), TokenRelayNotSet());
    if (amountWei % 1e10 != 0) revert Errors.InvalidAmount();
    (, uint256 reserve)
    = IHyperlinkTokenRelay(tokenRelay).getLstAccountingState();
    if (amountWei > reserve) revert Errors.InvalidAmount();
}
```

```
    IHyperlinkTokenRelay(tokenRelay)
        .claimUnstakedToVault(amountWei.to8Decimals());
        _recordHypeSpotDelta(int256(uint256(amountWei.to8Decimals())));
}
```

HyperLink: Resolved with [@2a9f4e9225 ...](#)

Zenith: Verified.

[L-3] Releasing the slashed reserve is possible before slashing is completed

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

Slashing can occur within 7 days and 12 hours after finalization.

```
claimWindowBuffer = 12 hours;

function applySlash(uint256 batchIndex, uint256 slashedExchangeRate)
    external onlyOwner {
    require(batchIndex < batches.length, InvalidBatch(batchIndex));
    Batch storage batch = batches[batchIndex];

    require(batch.finalizedAt > 0, InvalidBatch(batchIndex));

    require(block.timestamp < batchClaimableAt[batchIndex],
        CannotSlashBatchOutsideSlashWindow(batchIndex));
```

And releasing becomes available after 7 days.

```
function _releaseSlashedReserve(uint256 batchIndex)
    internal returns (uint256 released) {
    uint256 pending = batchPendingSlashReserveRelease[batchIndex];
    if (pending == 0) return 0;

    Batch memory batch = batches[batchIndex];
    if (block.timestamp < batch.finalizedAt + 7 days) return 0;
```

As a result, there is an overlapping period during which both operations are possible.'

Recommendations:

```
function _releaseSlashedReserve(uint256 batchIndex)
    internal returns (uint256 released) {
        uint256 pending = batchPendingSlashReserveRelease[batchIndex];
        if (pending == 0) return 0;

        Batch memory batch = batches[batchIndex];
        if (block.timestamp < batch.finalizedAt + 7 days) return 0;
        if (block.timestamp < batch.ClaimableAt[batchIndex]) return 0;
```

HyperLink: Resolved with [@2a9f4e9225 ...](#)

Zenith: Verified.

4.3 Informational

A total of 9 informational findings were identified.

[I-1] Update PR 90

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

In this PR, the priority fee reserver from PR 87 was also not considered like in PR 46. Also this PR should have no conflict between PR 91. And the `claimUnstakedToVault` function should be updated like in PR 46.

Recommendations:

```
function claimUnstakedToVault(uint64 weiAmount)
    external onlyStakingVault whenNotPaused {
        if (amountEvm > $.totalReservedLstWithdrawalHype) revert Errors.
            ClaimExceedsReservedLst();
        $.totalReservedLstWithdrawalHype -= amountEvm;

        $.totalReservedLstWithdrawalHype =
            amountEvm ≥ $.totalReservedLstWithdrawalHype ? 0 : $.totalReservedLst
                WithdrawalHype - amountEvm;
    }
```

And update the base branch of PR 90 and consider its relationship with PRs 87 and 91. Or close this PR.

HyperLink: Acknowledged. We are closing the pending-inclusive NAV / shadow-floor

branch for launch. The launch path keeps reserved-only live backing and avoids reintroducing the shadow staking floor.

[I-2] PR 87 and PR 91 should be considered in PR 46

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

In PR 46, the priority fee reserve from PR 87 was not considered.

```
function stakingAccountBalance() public view returns (uint256) {
    if (address(tokenRelay) == address(0)) revert TokenRelayNotConfigured();
    if (block.number <= tokenRelay.getLastLstAccountingChangeBlockNumber())
        revert StaleStakingAccountBalance();
    PrecompileLib.DelegatorSummary memory delegatorSummary
    = PrecompileLib.delegatorSummary(address(tokenRelay));
    uint256 l1Balance = _l1StakingBalance(delegatorSummary);
    (uint256 shadowBalance, uint256 reservedLstWithdrawalHype)
    = tokenRelay.getLstAccountingState();
    uint256 pendingWithdrawalBalance
    = delegatorSummary.totalPendingWithdrawal.to18Decimals();
    uint256 maturedUnclaimedBalance = 0;
    if (reservedLstWithdrawalHype > pendingWithdrawalBalance) {
        maturedUnclaimedBalance = reservedLstWithdrawalHype
        - pendingWithdrawalBalance;
    }

    uint256 l1PlusMaturedBalance = l1Balance + maturedUnclaimedBalance;
    return l1PlusMaturedBalance > shadowBalance ? l1PlusMaturedBalance :
    shadowBalance;
}
```

This PR should also have no conflicts with PR 91.

Recommendations:

Update the base branch of PR 46 and consider its relationship with PRs 87 and 91. Or close this PR.

HyperLink: Acknowledged. We are closing this branch for launch instead of rebasing it against PR87/PR91. The launch path keeps strict reserved-only accounting.

[I-3] 1e18 can be returned when the total supply is 0

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

When the total supply is 0, 1e18 can be returned as the exchange rate regardless of the remaining balance.

```
function exchangeRate() public view returns (uint256) {
    // After a complete wind-down, one HyperCore wei of residual HYPE can
    remain; treat it as zero.
    if (totalSupply == 0 && balance <= Converters.HYPERCORE_HYPE_UNIT) {
        return 1e18;
    }
}
```

Recommendations:

```
function exchangeRate() public view returns (uint256) {
    // After a complete wind-down, one HyperCore wei of residual HYPE can
    remain; treat it as zero.
    if (totalSupply == 0 && balance <= Converters.HYPERCORE_HYPE_UNIT) {
        if (totalSupply == 0) {
            return 1e18;
        }
    }
}
```

HyperLink: Acknowledged. We intentionally did not apply the literal `if (totalSupply == 0) return 1e18` recommendation because that would ignore meaningful zero-supply backing and let the next depositor capture it. Instead, `exchangeRate` returns 1e18 only for bounded HyperCore dust (`balance <= HYPERCORE_HYPE_UNIT`); non-dust zero-supply backing

still uses virtual-offset pricing to preserve donation/prefund protection.

[I-4] Unnecessary check in the _batchClaimableHypeAmount function

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

When the _batchClaimableHypeAmount function is called, refreshedRate is already greater than snapshotExchangeRate.

```
function _refreshCurrentBatchSnapshot() internal {
    uint256 refreshedRate = exchangeRate();
    if (refreshedRate <= batch.snapshotExchangeRate) return;
    batchClaimableHypeProcessed[currentBatchIndex]
    = _batchClaimableHypeAmount(currentBatchIndex, refreshedRate);
}
```

Therefore, the check below is not needed.

```
function _batchClaimableHypeAmount(uint256 batchIndex, uint256 rate)
    internal view returns (uint256 total) {
    if (batchIndex < batches.length) {
        Batch storage batch = batches[batchIndex];
        uint256 effectiveRate = batch.slashed ? batch.slashedExchangeRate :
        batch.snapshotExchangeRate;
        if (rate == effectiveRate)
            return batchClaimableHypeProcessed[batchIndex];
    }
}
```


Recommendations:

```
function _batchClaimableHypeAmount(uint256 batchIndex, uint256 rate)
    internal view returns (uint256 total) {
    if (batchIndex < batches.length) {
        Batch storage batch = batches[batchIndex];

        uint256 effectiveRate = batch.slashed ? batch.slashedExchangeRate :
            batch.snapshotExchangeRate;
        if (rate == effectiveRate) return batchClaimableHypeProcessed[
            batchIndex];
    }
}
```

HyperLink: Acknowledged. We are closing/deferring the batch snapshot-refresh branch for launch. The _batchClaimableHypeAmount path will not be included in the launch scope.

[I-5] Unnecessary check in the decreaseReservedLstWithdrawal function

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVault.sol](#)

Description:

When the decreaseReservedLstWithdrawal function is called, the released amount is already less than totalReservedLstWithdrawalHype.

```
function _releaseSlashedReserve(uint256 batchIndex)
    internal returns (uint256 released) {
    uint256 pending = batchPendingSlashReserveRelease[batchIndex];
    if (pending == 0) return 0;

    (, uint256 reserve) = tokenRelay.getLstAccountingState();
    uint256 needed = totalHypeProcessed > totalHypeClaimed ?
    totalHypeProcessed - totalHypeClaimed : 0;
    if (reserve <= needed) return 0;

    uint256 excess = (reserve - needed).stripUnsafePrecision();
    released = Math.min(pending, excess);
    if (released == 0) return 0;

    stakingVault.decreaseReservedLstWithdrawal(released);
}
```

Therefore, the check below is unnecessary.

```
function decreaseReservedLstWithdrawal(uint256 amountWei) external {
    if (msg.sender != stakingVaultManager) revert OnlyStakingVaultManager();
    require(tokenRelay != address(0), TokenRelayNotSet());
    if (amountWei % 1e10 != 0) revert Errors.InvalidAmount();
    ▷ (, uint256 reserve)
    = IHyperlinkTokenRelay(tokenRelay).getLstAccountingState();
```

```
▷   if (amountWei > reserve) revert Errors.InvalidAmount();  
   IHyperlinkTokenRelay(tokenRelay)  
       .claimUnstakedToVault(amountWei.to8Decimals());  
}
```

Recommendations:

```
function decreaseReservedLstWithdrawal(uint256 amountWei) external {  
    if (msg.sender ≠ stakingVaultManager) revert OnlyStakingVaultManager();  
    require(tokenRelay ≠ address(0), TokenRelayNotSet());  
    if (amountWei % 1e10 ≠ 0) revert Errors.InvalidAmount();  
    (, uint256 reserve) =  
        IHyperlinkTokenRelay(tokenRelay).getLstAccountingState();  
    if (amountWei > reserve) revert Errors.InvalidAmount();  
    IHyperlinkTokenRelay(tokenRelay)  
        .claimUnstakedToVault(amountWei.to8Decimals());  
}
```

HyperLink: Resolved with [@2a9f4e9225 ...](#)

Zenith: Verified.

[I-6] There is an unnecessary state fetch in the _releaseSlashedReserve function

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

In the _releaseSlashedReserve function, the accounting state is fetched from the token relay and stored in the reserve variable. This means preReleaseReserve should be the same as reserve.

```
function _releaseSlashedReserve(uint256 batchIndex)
    internal returns (uint256 released) {
    ▷    (, uint256 reserve) = tokenRelay.getLstAccountingState();
        uint256 needed = totalHypeProcessed > totalHypeClaimed ?
        totalHypeProcessed - totalHypeClaimed : 0;
        if (reserve <= needed) return 0;

        uint256 excess = (reserve - needed).stripUnsafePrecision();
        released = Math.min(pending, excess);
        if (released == 0) return 0;

        uint256 reserveAfterRelease = reserve - released;
        PrecompileLib.DelegatorSummary memory summary
        = PrecompileLib.delegatorSummary(address(tokenRelay));
        if (summary.totalPendingWithdrawal.to18Decimals() > reserveAfterRelease)
            return 0;

    ▷    (, uint256 preReleaseReserve) = tokenRelay.getLstAccountingState();
    }
```

Recommendations:

```
function _releaseSlashedReserve(uint256 batchIndex)
    internal returns (uint256 released) {
        (, uint256 preReleaseReserve) = tokenRelay.getLstAccountingState();
        uint256 preReleaseReserve = reserve;
    }
```

HyperLink: Resolved with [@2a9f4e9225 ...](#)

Zenith: Verified.

[I-7] Pending-inclusive NAV is redundant under normal accounting

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

`stakingAccountBalance()` now includes `Math.max(delegateSummary.totalPendingWithdrawal.to18Decimals(), reserved)` when computing LST backing. This changes the expression from reserved-only accounting to a pending-or-reserved accounting rule.

Under normal operation, `totalPendingWithdrawal ≤ reserved` should hold. `withdrawStake()` increments the relay's reserved counter when it initiates an exit, HyperCore's `totalPendingWithdrawal` decreases when the exit matures, and `claimUnstakedToVault()` only subtracts from reserved after the exit is mature and the funds are claimed. In that consistent state, `pendingOrReserved` collapses to reserved and should not inflate backing beyond protocol-owned withdrawal reserves.

This makes the new pending-inclusive semantics mostly superfluous during ordinary protocol operation: the additional `totalPendingWithdrawal` branch only changes behavior when the relay's reserved withdrawal accounting is inconsistent with HyperCore pending state, as discussed in issue [zenith-security/2026-05-hyperlink-brainstorming#36](#). In the expected state, the change should be equivalent to using reserved directly.

Recommendations:

Document that `Math.max(totalPendingWithdrawal, reserved)` is intended as a drift-tolerant fallback rather than normal NAV semantics. If this fallback is not needed, keep the simpler reserved-only accounting path and treat `totalPendingWithdrawal > reserved` as an invariant violation that should be monitored and reconciled.

HyperLink Acknowledged. We are closing the pending-inclusive NAV / shadow-floor branch for launch. The launch path keeps reserved-only live backing and avoids reintroducing the shadow staking floor.

[I-8] Priority-fee reserve attribution can mask concurrent undelegated drift

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [HyperlinkTokenRelay.sol](#)

Description:

The priority-fee reserve is tracked in `priorityFeeReserveHype`, while HyperCore exposes only the aggregate staking-account undelegated balance. The implementation handles this by subtracting `min(undelegated, priorityFeeReserveHype)` from LST backing in `stakingAccountBalance()` and `verifyOrSyncTotalStakedHype()`, and by excluding the same inferred reserve from `delegateUndelegated()`.

This is a reasonable attribution rule as long as the LST staking account invariant holds: normal stake flow should atomically deposit and delegate, and normal unstake flow should atomically undelegate and withdraw. Under that invariant, undelegated staking-account HYPE should not be present except for the priority-fee reserve or explicitly detected drift.

The ambiguous case is a stale `priorityFeeReserveHype` concurrent with real non-reserve undelegated drift. If priority fees are spent while unrelated LST HYPE remains undelegated, the `min()` deduction can continue attributing the undelegated balance to the stale reserve and shadow the fee spend. This can understate LST backing, cause health checks to report unexpected negative drift, block ambiguous downward syncs, or make operational repair paths such as `delegateUndelegated()` reject otherwise available undelegated stake.

Recommendations:

Monitor unexpected failures in staking, unstaking, and `delegateUndelegated()` flows as potential signs of mixed priority-fee reserve and undelegated-drift state. If this occurs, reconcile the priority-fee reserve with `reconcileLstAccounting(1, actualReserve, reason)` before topping up the reserve or repairing undelegated LST stake.

HyperLink Acknowledged. This is operational monitoring/reconciliation guidance, not a launch code-change blocker. We will monitor unexpected staking, unstaking, and `delegateUndelegated` failures as mixed reserve/drift signals and reconcile the priority-fee reserve before repair or top-up operations.

[I-9] Unbounded backward scan in batch refresh enables gas DoS

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [StakingVaultManager.sol](#)

Description:

The new `_batchClaimableHypeAmount` helper walks the `withdrawQueue` linked list backward from `lastProcessedWithdrawId`, accumulating per-withdraw stripped HYPE for every processed, non-cancelled withdrawal in the current batch.

```
function _batchClaimableHypeAmount(uint256 batchIndex, uint256 rate)
    internal view returns (uint256 total) {
    uint256 withdrawId = lastProcessedWithdrawId;
    while (withdrawId != 0) {
        Withdraw storage withdraw = withdraws[withdrawId];
        if (withdraw.batchIndex < batchIndex) break;
        if (withdraw.batchIndex == batchIndex && withdraw.cancelledAt == 0) {
            total += _hlHYPETOHYPE(withdraw.hlhypeAmount,
rate).stripUnsafePrecision();
        }

        (bool prevNodeExists, uint256 prevNodeId)
= withdrawQueue.getPreviousNode(withdrawId);
        if (!prevNodeExists) break;
        withdrawId = prevNodeId;
    }
}
```

The `_refreshCurrentBatchSnapshot` invokes this helper whenever the live exchange rate exceeds the stored batch snapshot, and that refresh is reached from both `processBatch()` and `finalizeBatch()`. Each loop iteration consumes a non-trivial amount of gas, so a sufficiently large current batch makes the unavoidable refresh on the next upward rate movement exceed the block gas limit, bricking `processBatch` and `finalizeBatch` until the operator unwinds the batch with `resetBatch` / `finalizeResetBatch`.

Recommendations:

Set a reasonable minimum `withdrawal amount` to prevent attackers from adding dust `withdrawal requests`.

HyperLink: Acknowledged. We are closing/deferring the batch snapshot-refresh branch for launch. The `_batchClaimableHypeAmount` path will not be included in the launch scope.